Beyond Coverage: Automatic Test Suite Augmentation for Enhanced Effectiveness using Large Language Models

ZEYU LU, Nanjing University, China
PENG ZHANG*, Nanjing University, China
YUGE NIE, Nanjing University, China
YIBIAO YANG*, Nanjing University, China
YUTIAN TANG, University of Glasgow, United Kingdom
CHUN YONG CHONG, Monash University Malaysia, Malaysia
YUMING ZHOU*, Nanjing University, China

Large Language Models (LLMs) have gained significant traction in software engineering for automating tasks such as unit test generation. Most existing studies prioritize code coverage as the primary metric for enhancing test suite effectiveness. However, prior research has shown that although code coverage can reach approximately 80%, the mutation score, which generally exhibits a stronger correlation with defect detection effectiveness, attains only about 35%. This gap highlights the need to enhance test suite effectiveness guided by mutation score rather than code coverage. Recent studies, including *MuTAP* and MutGen, explored the use of survived mutants to enhance test suite effectiveness. However, their evaluations were limited to simple standalone methods that rely on built-in functions and standard libraries. Non-standalone methods, which depend on other classes and involve complex user-defined types, are more intricate and commonly found in real-world projects. The limited contextual information and basic repair mechanisms in their prompt designs make it unclear whether their performance can generalize to non-standalone methods. Moreover, the two studies rely on existing language-specific, rule-based mutation techniques, which require specific configurations and incur additional costs when adapting to other programming languages.

To bridge this gap, we propose a novel, fully automatic LLM-based approach to enhance test suite effectiveness, guided by survived mutants. The approach augments initial test suites by integrating mutation testing with test case generation. It takes focal method information as input and generates test cases targeting survived mutants identified from applying the initial test suites. Our approach incorporates multiple prompt techniques, rich contextual information, and an advanced repair mechanism to effectively generate test cases for non-standalone methods. The evaluation covers 1,035 focal methods, categorized as standalone or non-standalone. On average, the mutation score increases by 16.04% for standalone methods and 8.11% for non-standalone methods. We validate the practical impact of augmented test suites in LLM-based code generation. After test suite augmentation, pass@1 decreased by 0.3152 and 0.1772 on average for standalone and non-standalone methods, respectively, indicating the effectiveness of our approach in reducing false positives caused by insufficient test cases in code generation evaluation.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

*Corresponding authors.

Authors' Contact Information: Zeyu Lu, Nanjing University, Nanjing, China, zeyulu@smail.nju.edu.cn; Peng Zhang, Nanjing University, Nanjing, China, dz1833034@smail.nju.edu.cn; Yuge Nie, Nanjing University, Nanjing, China, yuge_nie@smail.nju.edu.cn; Yibiao Yang, Nanjing University, Nanjing, China, yangyibiao@nju.edu.cn; Yutian Tang, University of Glasgow, Glasgow, United Kingdom, yutian.tang@glasgow.ac.uk; Chun Yong Chong, Monash University Malaysia, Subang Jaya, Malaysia, chong.chunyong@monash.edu; Yuming Zhou, State Key Laboratory of Novel Software Technology, Nanjing University, Nanjing, China, zhouyuming@nju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2475-1421/2026/4-ART143
https://doi.org/10.1145/3798251

Additional Key Words and Phrases: Large Language Models, Mutant Generation, Survived Mutants, Code Generation, Test Suite Augmentation

ACM Reference Format:
Zeyu Lu, Peng Zhang, Yuge Nie, Yibiao Yang, Yutian Tang, Chun Yong Chong, and Yuming Zhou. 2026. Beyond Coverage: Automatic Test Suite Augmentation for Enhanced Effectiveness using Large Language Models. *Proc. ACM Program. Lang.* 10, OOPSLA1, Article 143 (April 2026), 29 pages. https://doi.org/10.1145/3798251

1 Introduction

Unit testing plays a fundamental and crucial role in ensuring software quality. While unit tests can be generated manually, this process requires significantly more human effort. Over the years, numerous techniques for automatically generating test cases have been proposed. These techniques can be categorized into random testing (e.g., Randoop [59]), constraint-based testing (e.g., KLEE [16]), and search-based testing (e.g., EvoSuite [27]). Additionally, some studies generate test cases using model checkers [64] or learning-based approaches [73].

Recently, large language models (LLMs) [5, 14, 91] have gained popularity and brought transformative changes to the software engineering community. LLMs for generating unit test cases have the advantage of statically generating test cases based on given focal methods and being easily adaptable to multiple programming languages. Several studies [82, 85] have explored the use of LLMs for test case generation, revealing that directly generating test cases with LLMs often results in low code coverage. To overcome these limitations, various approaches have been proposed, including chain-of-thought (CoT) prompting [85] and retrieval-augmented generation (RAG) [90], to enhance the code coverage of LLM-generated test cases. Other approaches [7, 33, 47, 51, 61, 86] focus on generating test cases using feedback from code coverage information, targeting uncovered branches or paths in the focal methods to improve code coverage. With these enhancement techniques, previous studies [18, 33, 69, 78, 85, 90] have shown significantly improved code coverage of LLM-generated test cases.

However, most research on LLM-based test case generation focuses primarily on enhancing code coverage. Two commonly used metrics for assessing the defect detection effectiveness of a test suite are code coverage and mutation score. Code coverage measures the proportion of code elements, such as statements and branches, executed by a test suite relative to the total number of code elements. Mutation score, on the other hand, evaluates the proportion of mutants (artificially introduced variations of the program under test) that are killed by a test suite relative to the total number of generated mutants. Pan et al. [61] measured the mutation score achieved by LLM-generated test cases for Java projects and found that the mutation score ranged from 2% to 35%, even though the achieved statement coverage and branch coverage could reach nearly 80%. Compared to code coverage, most studies [12, 43] show that mutation score correlates more strongly with the defect detection effectiveness of test suites. Consequently, the mutation score, computed from the generated mutants, is a more effective metric for enhancing the effectiveness of test suites [63, 65, 67].

The need to enhance test suite effectiveness is recognized in datasets proposed for evaluating code understanding [34, 44, 54, 80] and code generation [24, 36, 44, 80, 87] tasks, where LLMs help reduce extensive human effort. Datasets in which focal methods are categorized as standalone (self-contained, invoking or accessing built-in functions or standard libraries) or non-standalone (with project-level or external API dependencies). In code generation tasks, test cases play a crucial role in ensuring the correctness of the generated code. However, many datasets [49, 50] highlight the ineffectiveness of test suites in evaluating code generation by LLMs. The evaluation of generated code may yield false positives, in which incorrect implementations are deemed correct due to

insufficient test cases. A previous study [50] reported that the estimated false positive rate of the HumanEval dataset [17] is 30%, while that of the APPS dataset [37] is considerably higher at 60%.

Efforts have been made to augment existing test suites in datasets used to evaluate code generation capabilities. In previous studies, initial test suites were typically constructed from simple human-written or programming competition problems, and additional tests were generated to reduce false positives. These studies commonly generate new test cases by executing newly created inputs and deriving valid outputs through consistency checks across multiple code samples. For example, AlphaCode [50] and EvalPlus [52] rely on mutating test inputs to generate additional test cases. Similarly, TACO [49] uses LLMs to supplement test cases by generating additional inputs for correct code samples and capturing their corresponding outputs. This approach works well for standalone methods with simple inputs and outputs, such as primitive types, strings, or collections. However, it is hardly applicable to datasets like CoderEval [84] and ComplexCodeEval [25], which contain non-standalone methods from real projects. Such methods often involve complex types (e.g., user-defined classes or interfaces within a project), making it difficult to generate new inputs and corresponding outputs in the same way. For example, a non-standalone method may have a test case with an assertion such as: *assertThatThrownBy(() -> ServiceParser.getServiceInstanceName(null)).isInstanceOf(NullPointerException.class);* Moreover, these studies lack general guidance for enhancing the sufficiency of test cases. For example, the TACO augmentation approach produced an average of over 200 tests, but this number is not universally applicable as guidance across all code generation tasks. Test suite size is an unrealistic objective for constructing test suites [19]. On the contrary, test adequacy criteria are considered a stopping rule that determines whether sufficient testing has been conducted and measures testing adequacy with respect to software testing requirements [92]. Code coverage or mutation score, measured based on test adequacy criteria, are widely deemed as proxies for test suite effectiveness [8, 12, 29–31, 43].

Enhancing test suite effectiveness guided by mutation score is more advantageous, as mutation score is generally a more reliable indicator of defect detection effectiveness than code coverage, and test suite size is an unrealistic objective for test suite construction. Recent studies, such as *MuTAP* [21] and MutGen [77], generate test cases targeting survived mutants to improve the mutation score. However, the two studies suffer from three limitations:

(L1) Insufficient evaluation. The focal methods used in *MuTAP* and MutGen are standalone and relatively simple, whereas real-world projects commonly involve non-standalone methods. Yu et al. [84] report that non-standalone methods account for more than 70% of methods in 100 popular Java and Python GitHub projects. Since focal methods in practical projects are typically non-standalone and more complex, the performance of *MuTAP* and MutGen on real-world projects remains unclear.

(L2) Simplistic prompt design. Compared with standalone methods, testing non-standalone methods may require mocks or stubs and contextual information from the entire project. The simplistic prompt design of *MuTAP* and MutGen, characterized by limited contextual information and a basic repair mechanism, could constrain their applicability to non-standalone methods.

(L3) Constrained adaptability. The mutation testing in *MuTAP* and MutGen is conducted as a separate process that relies on existing rule-based mutation tools, which are highly syntax-dependent and demand specific configuration. This reliance incurs significant costs when adapting to other programming languages.

In this study, we propose a fully automatic LLM-based approach guided by survived mutants that enhances the effectiveness of test suites. Our evaluation covers both standalone and non-standalone methods, filling the gap of prior studies that lack real-world project evaluation **(addressing L1)**. The approach combines mutation testing with test suite augmentation. It operates by taking information

about the focal method as input, while the survived mutants derived from the initial test suite further guide the test case generation. First, we use LLMs to generate mutants for focal methods. The code understanding capabilities of LLMs enable mutations to be conducted at the semantic level rather than the syntactic level. The mutant generation approach is designed to cover various behaviors of a focal method and to produce survived mutants. The approach requires neither training nor fine-tuning, is language-independent, and is readily adaptable for multi-language mutant generation **(addressing L3)**. Next, the survived mutants are used to guide the test case generation. To maximize the number of test cases that kill survived mutants, we employ multiple techniques, including CoT and RAG, during the test suite augmentation process. Class-level and project-level contextual information is incorporated into the prompts to further support effective test suite augmentation. Static analysis guides LLMs in repairing test cases through error message parsing and annotations **(addressing L2)**. Existing studies [18, 78, 85] that utilize LLMs for test case generation typically generate test cases from scratch, regardless of whether an initial test suite exists. In contrast, our approach augments the initial test suite of a focal method, thereby enhancing its initial mutation score, which refers to the mutation score achieved by the initial test suite prior to augmentation, and could complement these studies. If no test cases are available for a focal method, existing LLM-based test case generation techniques can be employed to construct an initial test suite.

Our contributions can be summarized as follows:

- We propose a novel approach for generating mutants using LLMs tailored to test suite augmentation. The approach focuses on semantic-level mutation and is language-independent, requiring no training or fine-tuning.
- We introduce a fully automatic LLM-based approach to enhance the effectiveness of initial test suites guided by survived mutants. The average mutation score increases by 16.04% for standalone methods and by 8.11% for non-standalone methods. The results on GitHub projects (i.e., non-standalone methods) indicate the practical applicability of the proposed approach.
- The augmentation approach offers a way to alleviate the false positive problem in existing datasets for code generation tasks. After augmentation, the pass@1 decreased by 0.3152 and 0.1772 on average for standalone and non-standalone focal methods, respectively, indicating reduced false positives in code generation evaluation.

The remainder of the paper is organized as follows. Section 2 offers a review of the background and prior work. Section 3 describes the proposed approach for mutation testing and test suite augmentation. Section 4 details the experimental setup and presents the results for both the proposed approach and the baselines. Section 5 reports additional experimental results and discusses threats to validity. Section 6 offers concluding remarks.

2 Background and Related Work

2.1 Mutant Generation

Rule-based approaches, such as PIT [20], are widely used and rely on predefined mutation operators that are applied to specific program elements. Rule-based mutant generation approaches rely on predefined transformation rules, which inherently limit mutation diversity. Although such approaches are effective at producing structurally valid mutants, they may fail to generate semantic-level mutants and struggle to approximate realistic developer errors. Learning-based approaches generate mutants by leveraging knowledge learned from large corpora of real-world bugs during training [70, 74, 75].

Recently, pre-trained LLMs have gained significant popularity, and several studies have explored their use for mutant generation. The code understanding capabilities of LLMs enable the generation of semantic-level mutants that better reflect natural human error patterns. μBERT [22] uses CodeBERT [26] to generate mutants by performing code completion for masked tokens. Similarly, LLMorpheus [71] generates mutants by prompting LLMs to perform code completion for JavaScript code. Beyond using code completion to generate mutants, MetaMut [57] leverages LLMs to generate mutation operators, which are then used to mutate C programs, and LLMut [76] uses LLMs to generate mutants for a single line of code. These studies [22, 57, 71, 76] focus on generating mutants without considering the killability of the generated mutants. In contrast, Ibrahimzada et al. [39] proposed BugFarm, which focuses on generating hard-to-kill mutants. They utilize the attention mechanism of pre-trained transformer models to identify the least attended program locations, then prompt LLMs to generate mutants. However, BugFarm requires fine-tuning to determine the least attended locations, which incurs additional time and resource costs.

2.2 Effectiveness of LLM-Based Test Case Generation

Among traditional approaches, symbolic execution [13, 41] enables systematic path exploration for test case generation and offers theoretical formal guarantees: a satisfying assignment produced by an SMT (Satisfiability Modulo Theories) solver ensures that the corresponding input executes the target path in the program. However, symbolic execution faces significant challenges. Even with environment modeling, SMT solvers often struggle with path constraints involving non-primitive types or external interactions, such as file systems or networking. In addition, symbolic execution typically generates only test inputs and does not provide expected outputs or behavioral oracles. Although LLM-based test case generation may suffer from hallucinations and lacks such formal guarantees, LLMs' code understanding capabilities enable them to handle complex dependencies and infer plausible expected behaviors to form assertions. Numerous studies [7, 18, 21, 33, 47, 51, 61, 66, 68, 69, 78, 81–83, 85, 86, 90] have explored the use of LLMs for test case generation. We categorize these studies from the following perspectives:

Enhancing Pass Rate: The test cases generated by LLMs suffer from a low pass rate, primarily due to compilation and execution errors [85]. To improve pass rates, most studies [7, 18, 21, 33, 47, 61, 68, 69, 78, 85, 90] adopt various approaches to repair the generated test cases. The effectiveness of test suites is then measured based on the passed test cases.

Achieved Test Suite Effectiveness: Several studies, such as ChatUniTest [18], measure code coverage achieved after test case generation. Pan et al. [61] proposed Aster and evaluated the code coverage and mutation score of test cases generated by LLMs for Java and Python classes or modules. For their results on Java projects, the statement and branch coverage of the generated test suites could reach nearly 80%, while the mutation score reached a maximum of 35%. The significantly lower mutation score of the generated test suites highlights the need for further enhancement to improve their effectiveness.

Enhancing Test Suite Effectiveness: Existing studies employ various methods to improve test suite effectiveness.

(1) *Code Coverage.* To enhance code coverage, different approaches have been proposed, including method-invocation analysis [81], CoT prompting [85], the combination of LLMs and symbolic execution [66], focal method slicing [78], RAG [90], and reinforcement learning [86]. Additionally, Aster [61], TestART [33], TestCTRL [86], CoverUp [7], STRUT [51], and YATE [47] improve test suite effectiveness by relying on feedback from code coverage.

(2) *Mutation Score.* When evaluating test suite effectiveness, the mutation score has been shown to be a more reliable predictor of the defect detection effectiveness of test suites than code coverage [43]. To generate more effective test suites, it is preferable to adopt mutation score-guided test

case generation. *MuTAP* [21] and MutGen [77] utilize survived mutants to generate test cases. However, their effectiveness on complex, practical code remains unclear.

3 Approach

3.1 Overview

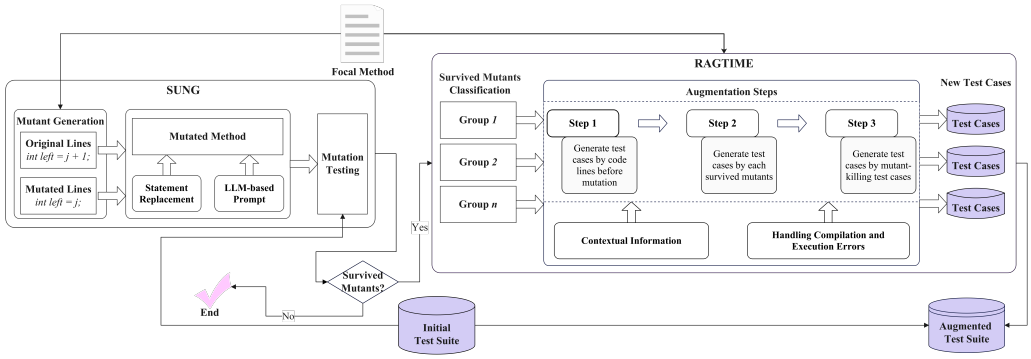


Fig. 1. Approach overview. The approach integrates mutation testing and test suite augmentation, where survived mutants from SUNG guide test case generation via RAGTIME.

Figure 1 presents an overview of our approach, which includes SUNG (LLMs for Semantic-level mUtaNt Generation) and RAGTIME (LLMs for suRvived mutAnts Guided Test suIte augMEntation). Given a focal method and the test cases that test it, we use SUNG to perform mutation testing. If any mutants survive, we use RAGTIME to generate new test cases to kill these survived mutants. The newly generated test cases are then merged with the initial test cases to form the augmented test suite. The entire process of our approach, including mutation testing and test suite augmentation, relies solely on in-context learning of LLMs and is fully automatic. Since LLMs are trained on multiple programming languages, our approach can be easily adapted to different programming languages.

3.2 SUNG

We propose SUNG, which uses LLMs to generate mutants based on program semantics and then conducts mutation testing. Algorithm 1 illustrates the procedure of SUNG.

3.2.1 Mutant Generation. We generate mutants tailored to the scenario to enhance the effectiveness of test suites based on survived mutants. First, we aim to generate mutants to enable the generation of test cases that exercise all possible behaviors of a focal method. Second, we aim to generate as many survived mutants as possible to guide the generation of additional test cases.

Approach. Mutant generation is conducted using six distinct sub-approaches (line 1 in Algorithm 1). While each sub-approach in SUNG operates largely independently, the six sub-approaches collectively function as an integrated whole.

Sub-Approach 1: Behavior Model. Mutant generation in this sub-approach is designed to encompass various program behaviors. The behavior of a program can be modeled as a *transition system*:

$$\mathcal{T} = (S, S_0, \Sigma, T)$$

where:

- S: The set of all possible *states* of the program.

Algorithm 1: Mutation Testing by SUNG

```
Input: FOCAL_METHOD
Output: SURVIVED_MUTANTS, INITIAL_MUTATION_SCORE
generated_mutants ← ApplyMutationSubapproachesSequentiallyWithLLMs() ;        // Repeated multiple times
survived_mutants ← [ ];
foreach mutant in generated_mutants do
    if mutant.Before.LOC == mutant.After.LOC and Match(mutant.Before, focal_method) then
        mutated_method ← StatementReplacement(mutant.Before, mutant.After, focal_method);
    else
        mutated_method ← GenerateMutatedMethodWithLLM(mutant.Before, mutant.After, focal_method);
    end
    focal_method ← mutated_method;
    status ∈ {killed, survived} ← ApplyTestSuite(focal_method);
    if IsSurvived(focal_method) then
        survived_mutants ← survived_mutants ∪ mutant;
    end
end
initial_mutation_score ← (len(generated_mutants) − len(survived_mutants)) / len(generated_mutants);
return [survived_mutants, initial_mutation_score];
```

- $S_0 \subseteq S$: The set of *initial states*.
- Σ: The set of *inputs*, also called the *alphabet*.
- $T \subseteq S \times \Sigma \times S$: The set of *transitions*, which describe how the program transitions between states.

We adopt CoT prompting, which uses LLMs to first generate a behavior model of the focal method, including its program states and transitions. For each program state and transition, we prompt LLMs to generate at least one mutant that causes the mutant's behavior to slightly deviate from the original behavior model.

Sub-Approach 2: RIP Theory. The RIP model [23, 53, 55] explains how test cases detect a mutant through (R)eaching the mutation point, (I)nfecting the program state, and then (P)ropagating the infection to an observable point checked by an oracle. To generate mutants that are hard for test cases to detect, we prompt LLMs to break the process of RIP, ensuring that:

(1) Even if the test cases reach the mutation point, they do not infect the program state.

(2) Even if the program state is infected, the infection does not propagate to the outputs.

The number of mutants generated in this approach is calculated as $\lceil \frac{\text{LOC}}{3} \rceil$, dynamically adjusting based on the lines of code (LOC) of the focal method.

Sub-Approach 3: Semantically Similar. Generating semantically similar but not equivalent mutants may introduce subtle behavioral changes that are harder for test cases to detect. We adopt CoT prompting, which prompts LLMs to first describe the behavior of the focal method, representing its semantics. The LLMs are then used to generate mutants that are semantically similar to the focal method but not semantically equivalent. The number of generated mutants in this approach is $\lceil \frac{\text{LOC}}{3} \rceil$.

Sub-Approach 4: Fault Localization. Targeting the suggested fault locations identified by LLMs in mutant generation may help improve test suite effectiveness by exercising fault-prone regions. We adopt CoT prompting, which leverages LLMs to identify locations where bugs or vulnerabilities might occur. For each identified location, we prompt LLMs to generate at least three mutants.

Sub-Approach 5: Method Invocation. Osman et al. [56] indicate that method invocations occupy a large proportion of bug-fix code changes evaluated across 717 open-source projects. For each method invocation in a focal method, we use LLMs to generate at least one mutant specifically tailored to the method invocation.

Sub-Approach 6: Boundary Conditions. Boundary conditions refer to edge cases or limit values at the boundaries of valid input or behavior in a program. Generating mutants for boundary conditions and detecting these mutants using test cases can help ensure that boundary conditions are correctly handled. We use LLMs to generate mutants that focus on all possible boundary conditions, specifically targeting values at the edges of the allowed ranges.

Discussion. The six sub-approaches are grounded in program behaviors (sub-approaches 1 and 3), defect detection research (sub-approaches 2), and program locations likely to expose errors (sub-approaches 4, 5, and 6). While not exhaustive, SUNG can be readily extended with other LLM-based strategies. Our work advances mutant generation by integrating diverse heuristics from software engineering and has the potential to inspire future research.

```
# Sub-Approach 1:
    {{prep_focal_class}}
    For the focal method {{focal_method_signature}}, first generate a behavior model of the focal method, including its program states and transitions.
    Then, for each program state and transition, generate at least one mutant that causes the mutant's behavior to slightly deviate from the original
behavior model.
# Sub-Approach 2:
    {{prep_focal_class}}
    The RIP theory explains how test cases detect a mutant:
    - (R)eaching the mutation point,
    - (I)nfecting the program state, and
    - (P)ropagating the infection to an observable point checked by an oracle.
    For the focal method {{focal_method_signature}}, you should generate at least {{non_empty_line_count/3}} mutants that are hard for test cases to detect,
based on the RIP theory, ensuring that:
    1) Even if the test cases reach the mutation point, they do not infect the program state.
    2) Even if the program state is infected, the infection does not propagate to the outputs.
# Sub-Approach 3:
    {{prep_focal_class}}
    For the focal method {{focal_method_signature}}, first describe the behavior of the focal method, which represents its semantics.
    Then generate at least {{non_empty_line_count/3}} mutants that are semantically similar to the focal method but not semantically equivalent.
# Sub-Approach 4:
    {{prep_focal_class}}
    For the focal method {{focal_method_signature}}, please identify locations where bugs or vulnerabilities might occur.
    For each identified location, generate at least 3 mutants to expose potential issues in the focal method.
# Sub-Approach 5:
    {{prep_focal_class}}
    For the focal method {{focal_method_signature}}, first identify all possible method invocations. The method invocation follows the format:
identifier(expression).
    Then, for each method invocation, generate at least one mutant that is specifically tailored to that invocation.
    Please ensure that the mutation of the method invocation in the focal method causes its behavior to deviate slightly from that of the original, non-
mutated version.
    The signatures of source methods defined in the same project and invoked by this focal method are as follows:
    {{matched_signatures}}
    Please take these method signatures into account when generating mutants.
# Sub-Approach 6:
    {{prep_focal_class}}
    Boundary conditions refer to edge cases or limit values at the boundaries of valid input or behavior in a program.
    Below are several scenarios for boundary conditions:
    For example (Numerical Values):
    Values just below and just above the boundaries (e.g., Integer.MIN_VALUE + 1 and Integer.MAX_VALUE - 1)."
    For example (Relational Operators):
    Replace equality checks (==) with inequality (>= or <=) for boundary values.
    Modify comparison operators while maintaining the same direction:
        Change <= to <, < to <=, >= to >, and > to >=.
        Avoid changing < to >, > to <, <= to >=, or >= to <=.
    Introduce off-by-one errors in the minimum or maximum value being tested (e.g., x < 10 becomes x < 9).
    For example (Array Index):
    Change the starting index from 0 to 1.
    Skip checking the upper boundary (array.length).
    Introduce off-by-one errors when iterating over arrays.
    For example (String Length):
    Test empty strings, single-character strings, and very large strings.
    Replace length checks with altered comparisons.
    For example (Logical Operators):
    Modify logical operators to change how boundary conditions are combined.
    For the focal method {{focal_method_signature}}, please generate mutants for all possible boundary conditions. The mutant generation is not restricted to
the above scenarios.
```

Fig. 2. Prompt template in SUNG.

Contextual Information. When generating mutants, we include the focal class in the prompt. The focal class, which contains the focal method, provides important contextual information (e.g., constructors and source methods) to help LLMs understand the focal method. We preprocess the focal class to restrict the inputs to the LLMs, allowing them to focus on the most essential content for mutant generation. The focal class is first preprocessed by removing all comments. The focal class is then preprocessed by retaining only the method signatures of the source methods, while preserving the import statements, constructors, and the focal method in their entirety. Fig. 2 presents the prompt template used in SUNG.

3.2.2 Mutation Testing. The output context limits of LLMs are generally fewer than 2,000 words [11], and each sub-approach may yield a large number of generated mutants. Since a focal method can contain many lines of code, output limits can easily be reached if LLMs are prompted to generate

````
"Before":
```java
Certificate[] certificateChain = getCertificatesFromString(clientIdentity.getCertificate());
```,
"After":
```java
// Mutant: Creating an invalid certificate chain
Certificate[] certificateChain = new Certificate[]{null};
```
````

Fig. 3. Example of the '*Before*' and '*After*' outputs from the cloud-security-services-integration-library-2.13.7 project.

the entire mutated method. For each generated mutant, SUNG is configured to output only the original line prior to mutation and mutated lines of a focal method. The output consists of two parts: '*Before*' and '*After*' (both stored in Markdown format), where '*Before*' represents the unmutated lines before applying mutation and '*After*' represents the mutated lines. A representative example from the cloud-security-services-integration-library-2.13.7 project is presented in Fig. 3.

However, due to LLMs' hallucinations or a lack of structural awareness of the source code, the '*Before*' and '*After*' sections may include unchanged lines or ellipses indicating omitted code, among other issues. To accurately map the mutation back to the focal method, we adopt two methods (lines 4-8): 1) If the code snippets in '*Before*' and '*After*' contain the same number of LOC, and the lines in the '*Before*' section can be matched within the focal method, we reconstruct the complete mutated focal method through statement replacement. 2) If the first method cannot be applied, we prompt the LLMs to generate the complete mutated focal method using the information contained in '*Before*' and '*After*'. For each generated mutant, we replace the original focal method with the complete mutated focal method (line 9) and apply the initial test suite to determine whether the mutant is killed or survived (lines 10). The initial mutation score is then calculated (line 15).

To facilitate LLMs' understanding of the semantic changes introduced by mutation, we present mutants in a format enriched with contextual information. We avoid directly using the '*Before*' and '*After*' parts, as they generally contain only the original line where the mutation is applied and the mutated lines. Specifically, for subsequent test case generation based on survived mutants, the mutants are presented to the LLMs in a format derived from applying the *git diff* command to the original and mutated focal methods. In the *git diff* output, the hunk header indicates the location of the changes within the focal method. Lines prefixed with '-' and '+' in the output represent the code removed and added to the focal method, respectively. Lines without prefixes are contextual lines that help locate the changes in the code. However, since the code lines prefixed with '-' and '+' are intermingled, we further process the *git diff* output to separate the code changes cleanly. The processed format includes two parts: 1) *Code snippet before mutation*: This part consists of the hunk header, the contextual lines, and the lines prefixed with '-'. 2) *Code snippet after mutation*: This part consists of the hunk header, the contextual lines, and the lines prefixed with '+'. In both parts, the markers '-' and '+' are replaced with a unified marker '*' to guide the LLM in focusing on the semantic changes, irrespective of whether they represent removals or additions.

3.3 RAGTIME

We propose RAGTIME to augment initial test suites guided by survived mutants. Algorithm 2 presents the procedure for enhancing the effectiveness of test suites. The algorithm takes the survived mutants and the initial mutation score from SUNG as inputs.

Algorithm 2: Test Suite Augmentation by RAGTIME

```
Input: SURVIVED_MUTANTS, INITIAL_MUTATION_SCORE
Output: KILLING_TEST_CASES, AUGMENTED_MUTATION_SCORE
survived_mutant_test_cases ← ∅;
foreach survived_mutant ∈ survived_mutants do
    survived_mutant_test_cases ← survived_mutant_test_cases ∪ (survived_mutant, false);
end
killing_test_cases ← ∅; symbol_list ← [];
classified_mutants ← ClassifyByClustering(survived_mutants);
foreach group in classified_mutants do
    Step 1;
    foreach survived_mutant, test_cases ∈ group do
        if test_cases ≠ false then
            group ← group \ (survived_mutant, test_cases);
        end
    end
    mutation_target_lines ← UnionOfCodeLinesBeforeMutation(group);
    pass_test_cases, fail_test_cases ← GenerateTestCasesByCodeLinesBeforeMutation(mutation_target_lines, symbol_list);
    pass_test_cases ← pass_test_cases ∪ RepairedTestCases(fail_test_cases, symbol_list);
    ExecuteTestCases(pass_test_cases, killing_test_cases, survived_mutant_test_cases);
    UpdateGroupWithGeneratedTestCases(group, survived_mutant_test_cases);
    Step 2;
    foreach survived_mutant, test_cases ∈ group do
        if test_cases ≠ false then
            continue;
        end
        pass_test_cases, fail_test_cases ← GenerateTestCasesByEachSurvivedMutants(survived_mutant, symbol_list);
        pass_test_cases ← pass_test_cases ∪ RepairedTestCases(fail_test_cases, symbol_list);
        ExecuteTestCases(pass_test_cases, killing_test_cases, survived_mutant_test_cases);
    end
    UpdateGroupWithGeneratedTestCases(group, survived_mutant_test_cases);
    Step 3;
    foreach survived_mutant and ∃ (newly_killed_mutants, new_test_case) ∈ survived_mutant_test_cases do
        pass_test_cases, fail_test_cases ← GenerateTestCasesByMutantKillingTestCases(symbol_list);
        pass_test_cases ← pass_test_cases ∪ RepairedTestCases(fail_test_cases, symbol_list);
        ExecuteTestCases(pass_test_cases, killing_test_cases, survived_mutant_test_cases);
    end
    UpdateGroupWithGeneratedTestCases(group, survived_mutant_test_cases);
end
augmented_mutation_score ← CalculateAugmentedMutationScore(killing_test_cases, initial_mutation_score);
Function ExecuteTestCases(pass_test_cases, killing_test_cases, survived_mutant_test_cases):
    foreach new_test_case ∈ pass_test_cases do
        test_class ← test_class ∪ new_test_case;
        newly_killed_mutants ← ExecuteTestCasesOnMutatedMethod(test_class);
        if Len(newly_killed_mutants) > 0 then
            killing_test_cases ← killing_test_cases ∪ new_test_case;
            survived_mutant_test_cases ← Refine(newly_killed_mutants, new_test_case);
        end
        test_class ← test_class \ new_test_case;
    end
return
Function UpdateGroupWithGeneratedTestCases(group, survived_mutant_test_cases):
    updated_group ← ∅;
    foreach (mutant, test_cases) ∈ group do
        foreach (survived_mutant, new_test_cases) ∈ survived_mutant_test_cases do
            if survived_mutant == mutant and new_test_cases ≠ test_cases then
                test_cases ← new_test_cases;
                break;
            end
        end
        updated_group ← updated_group ∪ (mutant, test_cases);
    end
    return updated_group;
return
```

3.3.1 Survived Mutants Classification. Since the mutated lines of different mutants may originate from the same unmutated line in a focal method, generating test cases for the same unmutated line can kill multiple related mutants, improving efficiency and reducing token consumption. We classify the code lines prior to mutation across all survived mutants. Mutants with identical code lines prior to mutation are grouped together. The classification further narrows the analysis scope of focal methods, particularly those with larger LOC, thereby facilitating test case generation by LLMs. Moreover, mutants within the same group are closely related, enabling test case generation based on similar mutants and their corresponding killing test cases.

```
1  public String getCommand(SummaryApplicationResponse application, AppDeploymentRequest request) {
2  * final boolean appHasCfEnv = hasCfEnv(request.getResource());
3  * return Stream.concat(Stream.of(application.getDetectedStartCommand()), request.getCommandlineArguments().stream())
4          .map( arg-> {
5  * int indexOfEquals = arg.indexOf("=");
6  * if(indexOfEquals > -1) {
7                  String key = arg.substring(0, indexOfEquals);
8                  String value = arg.substring(indexOfEquals);
9                  key = escapeChar(key, "(");
10                 key = escapeChar(key, ")");
11                 arg = key + value;
12             }
13 * return appHasCfEnv ? CfEnvConfigurer.appendCloudProfileToSpringProfilesActiveArg(arg) : arg;
14         })
15         .collect(Collectors.joining(" "));
16 }
```

Fig. 4. Focal method from spring-cloud-deployer-2.9.2 with 15 survived mutants classified into four groups across five mutation-targeted lines.

Algorithm 2 first classifies all survived mutants based on the similarity of the *code lines before mutation* (marked lines in the code snippet before mutation, line 6). Specifically, for each survived mutant, we use CodeBERT [26] to encode the code lines before mutation. We calculate the cosine similarity [60] of CodeBERT-generated embeddings, then apply agglomerative clustering [72], which dynamically determines the number of clusters, to classify all survived mutants. Fig. 4 presents the source code of a focal method from the project spring-cloud-deployer-2.9.2. All code lines prior to mutation across 15 survived mutants are marked with '*'. Although 15 mutants survived, they correspond to only 5 mutation-targeted lines, which are classified into 4 groups. Each group is represented by a distinct background color, with 3, 2, 2, and 8 mutants classified into the respective groups.

3.3.2 Augmentation Steps. After classifying the survived mutants, we adopt a three-step process to generate new test cases. The generation is based on each classified group.

Step 1: For survived mutants in a group, we identify all *code lines before mutation* and extract them from the focal method (line 14). LLMs are then prompted to generate 3 new test cases targeting these lines (line 15). The intuition behind this step is that mutations occur in these code lines; therefore, generating test cases targeting these code lines could potentially kill the mutants that modify them. Additionally, the extraction refers to focal methods, where code lines not targeted by mutation are replaced with empty lines. As a result, extracted lines in a group may be non-consecutive. Before prompting the LLM to generate test cases, we split extracted lines whenever more than one consecutive empty line appears to make the LLM focus on coherent code blocks.

Step 2: After Step 1, for the survived mutants in a group that still do not have test cases capable of killing them, we directly prompt the LLM to generate test cases for each survived mutant (line

24). The prompt includes the mutation information, i.e., the code snippets before and after mutation. We use CoT prompting, enabling the LLMs to first describe the change in program behavior caused by the mutation, and then generate 3 new test cases to detect it.

Step 3: After Step 2, for the survived mutants in a group that still do not have test cases capable of killing them, we adopt RAG, i.e., generating test cases based on those that successfully killed other mutants. Specifically, for each survived mutant, we prompt the LLM to generate 3 new test cases by imitating the newly generated test cases that were effective in killing mutants within the same group (line 31). The intuition here is that the groups are classified based on the similarity of mutants, and imitating the killing test cases generated in Step 1 or Step 2 can help produce test cases that target the non-killed mutants. To avoid exceeding the maximum input length of LLMs, we include at most 10 mutants and their corresponding killing test cases in each iteration, and iteratively add the remaining ones (if they exist) in subsequent prompts.

The generated test cases in each step are executed to determine whether each test case passes or fails on the focal method. The failed test cases then go through a repair process, detailed in the next subsection. If a failed test case is successfully repaired, it is added to the passed test cases (line 16, line 25, and line 32). For each test case in the passed test cases, the test case is merged into the existing test class (line 40). The merged test class is then executed against each survived mutant. If the test case successfully kills one of the survived mutants, it is added to *killing_test_cases*, which stores all generated killing test cases (lines 42-43). Regardless of whether the test case kills a survived mutant, it is removed from the test class (line 46). The generated killing test cases are dynamically updated in *survived_mutant_test_cases* (line 44) and each *group* (lines 49–61) to prevent the repeated generation of test cases for a mutant that already has test cases capable of killing it (lines 9-13, lines 21-23, and line 30). The entire process ends when all classified group iterations are complete.

```
# Step 1:
    The focal class is as follows:
    {{prep_focal_class}}
    The test class is as follows:
    {{prep_test_class}}
    The test methods {{test_case_names}} in the test class test the focal method {{focal_method_signature}} in the focal class.
    For the following lines of code in the focal method:
    {{code_lines_before_mutation}}
    First, describe the functionality of these lines in the focal method (using natural language only), then generate 3 new JUnit test cases that specifically
test them. The newly generated test cases will be added to the existing test class.
    When generating test cases, make full use of the current test class.
    When generating test cases, refer to how the existing test methods call the focal method.
    Specifically, pay attention to the parameter types of both the constructors and the focal method in the focal class.
    In the focal class, the private methods are: {{private_method_names}}, the private classes are: {{private_class_names}}. Private methods cannot be directly
invoked, and private classes cannot be directly instantiated.
    Please use the symbols that appear in both the focal class and the test class. Avoid using the following symbols: {{symbols}}
# Step 2:
    The focal class is as follows:
    {{prep_focal_class}}
    The test class is as follows:
    {{prep_test_class}}
    The test methods {{test_case_names}} in the test class test the focal method {{focal_method_signature}} in the focal class.
    {{mutation_info}}
    The lines of code marked with '*' indicate that the code has been modified. The lines of code without the '*' mark help to localize the changes in the code.
    Please describe the change in program behavior caused by the mutation (using natural language only), then generate at least 3 new JUnit test cases, to be
added to the existing test class, that can detect this change in program behavior in the focal method {{focal_method_signature}}.
    When generating test cases, make full use of the current test class.
    When generating test cases, refer to how the focal method is called in existing test methods that test it.
    Specifically, pay attention to the parameter types of both the constructors and the focal method in the focal class.
    In the focal class, the private methods are: {{private_method_names}}, the private classes are: {{private_class_names}}. Private methods cannot be directly
invoked, and private classes cannot be directly instantiated.
    Please use the symbols that appear in both the focal class and the test class. Avoid using the following symbols: {{symbols}}
# Step 3:
    The focal class is as follows:
    {{prep_focal_class}}
    The test class is as follows:
    {{prep_test_class}}
    The following mutant was generated by mutating the focal method {{focal_method_signature}} in the focal class.
    {{mutation_info}}
    The lines of code marked with '*' indicate that the code has been modified. The lines of code without the '*' mark help to localize the changes in the code.
    However, this mutant cannot be detected by existing test cases. There are, however, similar mutants that test cases can detect:
    {{killing_message}}
    Please generate at least 3 test cases to detect the undetected mutant by imitating the test cases that detect similar mutants. The generated test cases are
to be added to the existing test class.
    When generating test cases, make full use of the current test class.
    When generating test cases, refer to how the focal method is called in test cases that could detect similar mutants.
    Specifically, pay attention to the parameter types of both the constructors and the focal method in the focal class.
    In the focal class, the private methods are: {{private_method_names}}, the private classes are: {{private_class_names}}. Private methods cannot be directly
invoked, and private classes cannot be directly instantiated.
    Please use the symbols that appear in both the focal class and the test class. Avoid using the following symbols: {{symbols}}
```

Fig. 5. Prompt template used for test case generation across the three augmentation steps.

Contextual Information. In test case generation, we include the focal class and its corresponding test class in the prompt. Both classes are first preprocessed by removing comments. The focal class is then handled in the same way as in SUNG. For the test class, we remove methods that do not test the focal method, while retaining those that test the focal method along with any related helper methods or classes. Including the test class in the prompt guides the LLMs to reference the focal method's invocation, ensuring that the generated test cases call the focal method correctly and providing important contextual information to help generate more passing test cases. The prompt template used for test case generation across the three augmentation steps is shown in Fig. 5.

3.3.3 Handling Compilation and Execution Errors. The generated test cases may contain compilation and execution errors, leading to the failure of executing the test cases on a focal method. To generate more passing test cases, we adopt two approaches: *compilation error prevention* and *test case repair*.

Compilation Error Prevention. Compilation error prevention aims to ensure the correct invocation of the focal method in the generated test cases. 1) *Access Error.* In test case generation, we perform static analysis to identify private subclasses and methods within the focal class. The results of this analysis are incorporated into the prompt to prevent direct access to these private subclasses and methods. 2) *Symbol Resolution Error.* Similar to previous studies [18, 69, 85], the main type of compilation error we encounter is symbol resolution errors during both test case generation and repair. If the parsed error message contains "*cannot find symbol*," we extract the undefined symbols. During test case generation and repair, we maintain a list of unique symbols, which is incorporated into the prompt to prevent subsequently generated test cases from including these undefined symbols. This list is updated iteratively and dynamically throughout the generation and repair process.

```
[{
'file': 'cloud-security-services-integration-
      library-2.13.7/token-client/src/test/java/
      com/sap/cloud/security/mtls/
      SSLContextFactoryTest.java',
'line': 97,
'column': 25,
'message': 'cannot find symbol',
'symbol': 'class CertificateException',
'location': 'class com.sap.cloud.security.mtls.
      SSLContextFactoryTest'
}]
```

(a) Parsed errors.

```
public class SSLContextFactoryTest {
    ......
    @Test
    void createKeyStore_
         throwsOnInvalidCertificateString() {
        assertThatThrownBy(() -> {
            cut.createKeyStore(new
                 ClientCertificate("invalid-
                 certificate-string",
                 rsaPrivateKey, null));
        }).isInstanceOf(CertificateException.
             class); // @Error: cannot find
             symbol class CertificateException
    }
}
```

(b) Annotated test case.

Fig. 6. Example of parsed errors and the corresponding annotated test case.

Test Case Repair. If the generated test cases contain compilation or execution errors, we address these issues by collecting error messages and prompting the LLMs to repair the test cases accordingly. For a test case that requires repair, we parse the error message (Fig. 6(a)) to extract the test class path, the line number where the error occurs, and the detailed error description. The line number indicates the specific line in the generated test case where the error occurs. We then modify the test class by adding the error description as a comment at the end of the specified line. This annotated test case (Fig. 6(b)) is presented to the LLMs for test case repair.

In addition to the contextual information used in test case generation, we incorporate contextual information from within the same project [18, 66, 83, 85]. Specifically, for the declarations of dependent classes of the focal method and the signatures of methods invoked by the focal method, we

include these classes and methods if they are referenced in the parsed error message. Incorporating the definitions of classes and methods from the same project provides the LLMs with richer contextual information for test case repair.

```
    The focal class is as follows:
    {{prep_focal_class}}
    The test class is as follows:
    {{test_class_with_error_messages}}

# For Step 1 only:
    The test case {{test_case_name}} in the test class is specifically generated to test the following lines of code in the focal
method {{focal_method_signature}} in the focal class:
    {{code_lines_before_mutation}}
# For Steps 2 and 3:
    The test case {{test_case_name}} in the test class is specifically generated to test the mutation occurring in the focal method
{{focal_method_signature}} in the focal class.
    The mutation information that alters the focal method is:
    {{mutation_info}}
    The lines of code marked with '*' indicate that the code has been modified. The lines of code without the '*' mark help to localize
the changes in the code.

    However, I encountered errors after executing the test case, and the error message is explicitly marked (// @Error:) in the test
case.
    Please analyze the errors and repair the test case accordingly.
    When repairing the test case, refer to the information of the dependent classes and the signatures of source methods invoked by the
focal method within the same project:
    {{focal_method_dependency}}
    When repairing test cases, make full use of the current test class.
    Please use the symbols that appear in both the focal class and the test class. Avoid using the following symbols: {{symbols}}
```

Fig. 7. Prompt template used for test case repair.

The repair attempt for a test case is repeated up to three times. After each repair attempt, the test case is executed to validate its correctness. If the repaired test case is successfully executed in any of the three attempts, it is added to the *pass_test_cases*, and the repair attempts stop. Conversely, if a test case cannot be successfully repaired within three attempts, it is discarded. The repair process continues with the remaining test cases that require repair and stops if none exist. Fig. 7 presents the prompt template used for test case repair.

3.4 Non-Determinism of LLMs

One of the challenges in using LLMs is their non-determinism. Since this issue cannot be fully resolved at present [58], we follow a previous study [68] to address the problem through *repetitive runs*. Specifically, the process of mutant generation is repeated three times (line 1 in Algorithm 1), and the mapping of mutations back to the complete focal method is also repeated (line 7 in Algorithm 1). All mutants generated across these three iterations are combined to form the final set of mutants. We avoid excessive repetitions (e.g., 10) to mitigate the high cost associated with invoking the LLM API. For test suite augmentation, we prompt LLMs to generate three distinct test cases each time during both the generation and repair phases. The augmentation process is not repeated, as the three augmentation steps for each classified group of survived mutants, combined with iterating over all groups, help ensure comprehensive test case generation. Another reason is the significant token consumption and time required to generate and execute test cases against survived mutants.

4 Evaluation

4.1 Experimental Setup

4.1.1 Research Questions. We investigate the effectiveness of the proposed approach by answering the following research questions (RQs):

- **RQ1: (Characteristics of Generated Mutants).** How effective is SUNG at generating survived mutants to facilitate test suite augmentation?
- **RQ2: (Augmented Mutation Score).** How effective is RAGTIME in augmenting mutation scores for standalone and non-standalone methods?
- **RQ3: (Evaluation of Code Generation of LLMs).** How does test suite augmentation impact the evaluation of LLM-synthesized code by reducing false positives?

4.1.2 Baselines. For mutant generation, we use PIT [20] as the baseline. PIT is a rule-based mutant generation approach that has been widely used in prior studies [6, 31, 32, 40, 46, 89] and is also adopted in MutGen [77]. For test suite augmentation, we select EvoSuite as a baseline. EvoSuite has been widely adopted as a baseline in many test case generation studies [18, 33, 69, 78, 85, 90]. It aims to maximize multiple objectives, such as line coverage, branch coverage, weak mutation coverage, and output coverage, when generating test cases. We also include MutGen [77], the study most similar to ours that generates test cases for survived mutants, as another baseline. We do not include *MuTAP* [21] in the comparison, as MutGen is methodologically similar to it, with the primary distinction being the programming language. Moreover, *MuTAP* requires running test inputs on the program under test to obtain correct outputs during test case repair. This is suitable for standalone methods, whose inputs and outputs typically involve primitive types or strings, but is hardly applicable to non-standalone methods, which often involve complex types such as user-defined classes within a project.

4.1.3 Dataset. The existing datasets [10, 17, 37, 44, 49, 80] for code understanding and generation are typically derived from standalone programs, which are often sourced from programming competition websites like Codeforces. However, pragmatic code understanding and generation commonly involve non-standalone methods due to external dependencies [84]. Therefore, we select datasets comprising both standalone and non-standalone methods to evaluate the proposed test suite augmentation approach.

Dataset Constructed from Standalone Methods. Standalone source methods are generally simpler and relatively easier to generate test cases for. *MuTAP* [21] reports that its evaluations were conducted on methods with an average LOC of 11.5 in HumanEval [17] and 8.4 on Refactory [38]. In our study, we specifically select standalone methods with LOCs greater than or equal to 20. Excluding trivial standalone methods ensures evaluation on methods with sufficient structural complexity, which improves the comparability of different test case generation approaches and complements the evaluations of previous studies.

MBJP. MuTAP [21] used HumanEval in generating test cases. Athiwaratkun et al. [9] presents multi-language datasets include multilingual MBPP (MBXP) and HumanEval. However, the Java programs converted from HumanEval does not include canonical solutions written in Java, we use MBJP as our dataset (where 'J' stands for Java). There are a total of 974 programs in MBJP. Applying the criteria of LOC $\geq$ 20 and the presence of passing test suites results in a final set of 58 programs (with an average of 22.59 LOC).

LeetCode. We further construct a dataset from LeetCode problems. We select non-premium problems with a difficulty level of medium or hard, resulting in 2,096 problems that meet these criteria. From these, we randomly sample 1,000 problems. After sampling, we select focal classes from the top three ranked Java solutions. If the primary source method that solves the problem in the Solution class has $\geq$ 20 LOC, then the Solution class is designated as the focal class, and the primary source method is set as the focal method. This results in 468 problems (with an average of 29.95 LOC). The initial test suite consists of the test cases provided with each selected problem.

Dataset Constructed from Non-Standalone Methods. Datasets constructed from GitHub repositories are more complex. Dependencies among source methods and API invocations within a practical project generally make source methods non-standalone.

ComplexCodeEval. We use ComplexCodeEval [25], a dataset constructed from GitHub repositories. For the Java programming language, it contains 3,897 focal methods from 1,055 different projects. However, since ComplexCodeEval is constructed statically, we find that many focal methods cannot be successfully compiled or executed. We randomly selected 100 projects from all Java projects in ComplexCodeEval. Among these, 24 could be successfully compiled and executed on our machine.

In addition to the projects from ComplexCodeEval, we randomly selected three three Java EE projects (datafaker, ice4j, and openapi-diff) from prior studies on test case generation [18, 78, 90]. From these 27 projects, we selected focal methods based on two criteria: (1) The focal methods are not constructors or private methods. (2) At least one test method exists for the focal method. We employ Understand [4] for focal method selection, as it enables the identification of a greater number of focal methods with associated test methods compared to ComplexCodeEval, which relies exclusively on name matching. This results in 509 focal methods. The 27 projects incorporate test cases implemented in both JUnit 4 and JUnit 5. Detailed information on the 27 projects is provided in the supplementary material [3].

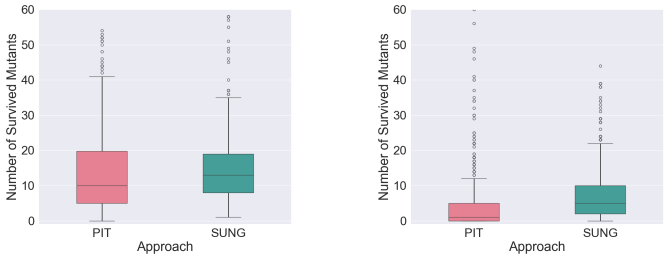
4.1.4 Implementation. Augmenting the test suites for all 1,035 focal methods can result in substantial token consumption. To balance cost and performance, we use DeepSeek-V3 for all tasks in SUNG and RAGTIME. Static analysis in SUNG and RAGTIME is conducted using the ANTLR v4 [62] Java API. In SUNG, we specify the mutation testing timeout to be 900 seconds per run throughout all experiments. The experiments were performed on a server with an Intel 48-core 2.30 GHz CPU, 126 GB RAM, and Ubuntu 22.04 LTS.

4.2 RQ1: Characteristics of Generated Mutants

Approach. We compare the number of survived mutants generated by SUNG with those generated by PIT. The survived mutants are identified by executing the initial test suites on their corresponding focal methods. PIT is configured to utilize all supported mutation operators for mutant generation.

Table 1. Statistics of Generated Survived Mutants

Focal Methods	Mean		Median		Win/Tie/Loss (%)	*p-value*
	PIT	SUNG	PIT	SUNG	PIT vs. SUNG	
Standalone	**15.82**	14.38	10.00	**13.00**	38.59/4.94/**56.46**	0.01312
Non-Standalone	5.80	**7.77**	1.00	**5.00**	28.88/9.23/**61.89**	8.91e-16



(a) Standalone Methods (b) Non-Standalone Methods

Fig. 8. Comparison of the number of survived mutants.

Results. Table 1 presents statistics on the survived mutants generated by both PIT and SUNG. The better values are highlighted in bold in Table 1. Fig. 8 presents the boxplot of the number of survived mutants, with each data point representing the number of survived mutants associated with a focal method. As shown in Table 1, the number of survived mutants for non-standalone

methods remains lower than for standalone methods. This can be attributed to the dependencies of non-standalone methods, which enable test cases to detect changes in a focal method more easily, as well as the larger number of accompanying tests. On average, the initial test suite size for standalone methods is 2.56, whereas the corresponding size for non-standalone methods is 2.85.

For standalone methods, SUNG yields a higher median number of survived mutants than PIT; however, its average number of survived mutants is slightly lower. This is because PIT can generate a significantly larger number of mutants for certain focal methods. For non-standalone methods, SUNG produces higher average and median numbers of survived mutants. Fig. 8(a) and Fig. 8(b) present boxplots with the y-axis maximum set to 60. This threshold omits focal methods for which the two approaches generate an excessively large number of mutants. In pairwise comparisons, SUNG produces more survived mutants than PIT in 56.46% of standalone methods and in 61.89% of non-standalone methods. We further apply the Wilcoxon signed-rank test to examine the statistical differences in the number of survived mutants generated by PIT and SUNG. As shown in Table 8, the *p-values* for standalone and non-standalone methods are 0.01312 and 8.91e-16, respectively, both below the 0.05 threshold. The negative *z-value* in the Wilcoxon signed-rank test indicates that SUNG produces a higher number of survived mutants.

PIT achieves a 100.00% mutation score on 20 standalone focal methods (3.80%), whereas SUNG achieves it on none (0.0%). A 100.00% initial mutation score indicates that no survived mutants were generated, rendering them unusable for subsequent test suite augmentation. PIT reaches a 100.00% mutation score on 127 non-standalone focal methods (24.95%), whereas SUNG achieves this on 55 focal methods (10.81%). Additionally, JaCoCo [2] reports 50.95% of standalone methods and 52.39% of non-standalone methods reaching 100% line coverage, with the latter excluding 28 methods for which JaCoCo failed to generate reports.

```
static RpcRequest spCursorOpen(String query, Collation collation, TransactionDescriptor
      transactionDescriptor) {
    ......
    return RpcRequest.builder() //
        .withProcId(RpcRequest.Sp_CursorOpen) //
        .withTransactionDescriptor(transactionDescriptor) //
        .withParameter(RpcDirection.OUT, 0) // cursor
        .withParameter(RpcDirection.IN, collation, query)
        .withParameter(RpcDirection.IN, resultSetScrollOpt)  // scrollopt
        .withParameter(RpcDirection.IN, resultSetCCOpt) // ccopt
        .withParameter(RpcDirection.OUT, 0) // rowcount
        .build();
}
```

(a) Focal method spCursorOpen() in project r2dbc-mssql-1.0.0.RELEASE.

```
return RpcRequest.builder().withProcId(RpcRequest.Sp_CursorOpen)
==>
return RpcRequest.builder().withProcId(-1).withProcId(RpcRequest.Sp_CursorOpen)

.withTransactionDescriptor(transactionDescriptor).withParameter(RpcDirection.OUT, 0)
==>
.withTransactionDescriptor(TransactionDescriptor.empty()).withParameter(RpcDirection.OUT, 0)
```

(b) Two generated mutants of the focal method spCursorOpen() by SUNG.

Fig. 9. Focal method spCursorOpen() and its mutants.

Compared to PIT, SUNG leverages the LLM's code understanding capabilities to generate mutants at the semantic level. Fig. 9(a) presents the source code and corresponding mutants of the focal method *spCursorOpen()*. For this method, PIT generates 1 mutant with 0 survived, whereas SUNG generates 91 mutants with 6 survived. In this case, PIT produces far fewer mutants, likely because

the code is complex and PIT cannot apply mutation operators to code segments that do not satisfy its application rules. In contrast, the mutants generated by SUNG are not constrained by the need to locate program elements where mutation operators can be applied. Fig. 9(b) illustrates two such mutants generated by SUNG, both of which could not be easily produced by traditional rule-based mutation approaches.

Answer for RQ1: For both standalone and non-standalone methods, SUNG generally produces more survived mutants, thereby enhancing the potential for test suite augmentation based on these mutants.

4.3 RQ2: Augmented Mutation Score

Approach. The survived mutants obtained from SUNG are further used for test suite augmentation. For EvoSuite and MutGen, the default test case generation setting starts from empty test suites, regardless of whether existing test cases are available. For consistency, test case generation for the baselines targets all mutants. As analyzed in the previous subsection, there are focal methods with a mutation score of 100%, meaning they have no survived mutants. Similar to MutGen, we eliminate these focal methods from test suite augmentation. This results in 980 focal methods (526 standalone and 454 non-standalone) out of a total of 1,035 focal methods. In addition, MutGen is configured to use the mutants generated by SUNG and the same LLM as RAGTIME throughout the experiments to ensure a fair comparison. We then compare the augmented mutation scores achieved by RAGTIME with the mutation scores of the baselines generated from empty test suites. To ensure a fairer comparison, we further compare RAGTIME with MutGen by configuring MutGen to generate test cases only for survived mutants. This comparison is conducted on a randomly sampled set of focal methods.

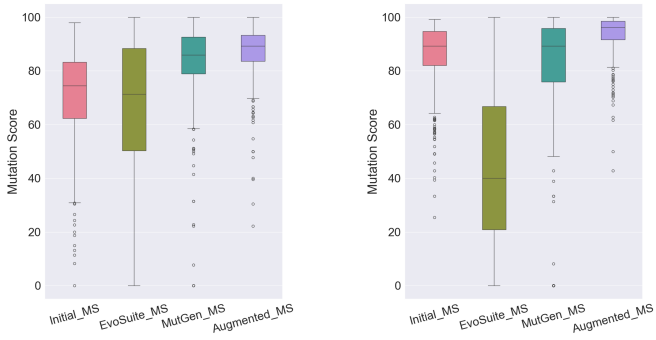
Table 2. Mutation Scores and Sizes of Generated Test Suites

Focal Methods	EvoSuite		MutGen		RAGTIME						
	MS	TSS	MS	TSS	Augmented MS	Initial MS	ΔMS	Augmented TSS	Initial TSS	ΔTSS	
Standalone	65.71*	6.80*	84.08	**29.04**	**87.37**	71.33	16.04	27.11	2.56	24.56	Mean
	71.31*	6.00*	86.00	**25.00**	**89.24**	74.49	14.75	22.00	3.00	19.00	Median
Non-Standalone	42.59*	6.24*	80.50	6.92	**94.05**	85.94	8.11	**12.65**	2.85	9.80	Mean
	40.00*	4.50*	89.24	4.00	**96.20**	89.29	6.91	**7.00**	2.00	5.00	Median

MS stands for Mutation Score. TSS denotes Test Suite Size. The ΔMS and ΔTSS columns indicate the differences between the augmented and initial mutation scores and test suite sizes, respectively.
MutGen and RAGTIME results on non-standalone methods are based on 27 projects, whereas EvoSuite results are derived from 19 projects and marked with an asterisk (*).

Results. Table 2 presents the average mutation scores and test suite sizes for test cases generated by EvoSuite, MutGen, and RAGTIME. The highest values are highlighted in bold in Table 2. Fig. 10 shows the boxplots of mutation scores achieved by EvoSuite, MutGen, and RAGTIME, along with the initial mutation score measured from the initial test cases. For non-standalone methods, we use Java 17 to execute the projects containing these methods. However, EvoSuite supports only up to Java 11, and eight projects require Java 17 for successful compilation. Following a prior study [78], we removed these eight projects, leaving 19 projects in the comparison.

Comparison. As shown in Table 2 and Fig. 10, both baselines perform better on standalone methods than on non-standalone methods. Notably, EvoSuite's mutation score generally fails to exceed the initial mutation score for non-standalone methods. Additionally, the augmented mutation score



(a) Standalone Methods (b) Non-Standalone Methods

Fig. 10. Mutation score comparison. Results for MutGen and RAGTIME on non-standalone methods use 27 projects; EvoSuite uses 19.

of RAGTIME surpasses the mutation score achieved by the baselines. On average, the augmented mutation score of RAGTIME surpasses that of EvoSuite by 21.66% and 51.32% for standalone and non-standalone methods, with the latter comparison based on focal methods from 19 projects. Similarly, the augmented mutation score of RAGTIME surpasses that of MutGen by 3.30% and 13.55% for standalone and non-standalone methods, respectively. Furthermore, RAGTIME attains a 100% augmented mutation score on 6.46% of standalone methods, whereas EvoSuite and MutGen achieve 8.56% and 7.98%, respectively. For non-standalone methods, RAGTIME similarly achieves a 100% augmented mutation score on 22.69% of methods (21.59% across 19 projects), compared with 1.70% and 10.49% for EvoSuite and MutGen. Achieving 100% mutation scores on a non-trivial proportion of focal methods further suggests that the mutants generated by SUNG contain a smaller proportion of equivalent mutants.

EvoSuite produces the smallest test suite size among the three approaches. Its design reduces redundancy by removing test cases while preserving the same test goals as the non-removed test suites [28]. However, EvoSuite also yields the lowest mutation score. Moreover, MutGen produces larger test suites than RAGTIME for standalone methods. MutGen retains all test cases that pass on a focal method, whereas RAGTIME keeps only those that improve the mutation score. For non-standalone methods, however, MutGen generates significantly smaller test suites compared with RAGTIME, mainly due to its much lower mutation score.

For the comparison of RAGTIME with MutGen restricted to survived mutants, the sampling strategy [48] is determined with a 95% confidence level and a 10% margin of error applied to the 980 focal methods, resulting in 88 sampled focal methods. These methods are proportionally distributed across standalone and non-standalone categories, with 47 and 41 focal methods randomly selected, respectively. RAGTIME kills, on average, 3.25% and 25.58% more survived mutants than MutGen for 47 standalone and 41 non-standalone methods, respectively.

Compared with EvoSuite, RAGTIME eliminates constraints related to Java version compatibility during compilation and thereby enhances its practical applicability. Compared with MutGen, RAGTIME performs better, particularly for non-standalone methods. This superiority may be attributed to several factors, including the incorporation of richer contextual information, the inclusion of existing test cases in prompts to help LLMs generate valid test cases that successfully

enter focal methods, the application of multi-step augmentation, and the use of a more sophisticated test case repair mechanism.

RAGTIME. As shown in Table 2, the average difference between the augmented and initial mutation scores is 16.04% for standalone methods and 8.11% for non-standalone methods, demonstrating the effectiveness of RAGTIME in enhancing the defect detection capability of test suites. However, the greater number of accompanying test cases in non-standalone methods (2.85 vs. 2.56 on average), which leads to higher initial mutation scores, together with the increased complexity of real-world projects, results in a smaller mutation score improvement compared to standalone methods.

While our approach outperforms the baselines, there are focal methods for which RAGTIME cannot generate test cases that kill survived mutants. In RAGTIME, a test case is stored only if it improves the mutation score. Therefore, even if a substantial number of test cases are generated for a focal method, none may succeed in killing the survived mutants, and these test cases are discarded. For standalone methods, 7.60% have no killing test cases. For non-standalone methods, the percentage is 25.33%, which is higher than for standalone methods. The main reason we identified is that the number of survived mutants is often very small—for example, just one or two. Another reason may be that non-standalone methods are more complex and have intricate dependencies, making it difficult to generate test cases that effectively target survived mutants. For these focal methods, we set the augmented mutation score as the initial mutation score. Furthermore, excluding focal methods without killing test cases yields an average mutation score increase of 17.36% for standalone methods and 10.86% for non-standalone methods.

Answer for RQ2: On average, the augmented mutation score achieved by RAGTIME surpasses the mutation score of EvoSuite by 21.66% for standalone methods and 51.32% for non-standalone methods, and exceeds that of MutGen by 3.30% and 13.55%, respectively.

4.4 RQ3: Evaluation of Code Generation of LLMs

Approach. The augmented test suites enhance the testing of focal methods. These enhanced suites can be employed to strengthen the evaluation of LLM-generated code, an area that supports software development and has attracted considerable research interest [42]. On the contrary, relying on insufficient test cases may lead to false positives and, consequently, incorrect conclusions. For code generation evaluation, the adopted focal methods serve as ground truth for LLM-synthesized code. Both the before- and after-augmentation test suites pass on these focal methods; conversely, any failures on the LLM-generated implementations indicate errors in the generated code.

To evaluate the generated code, we adopt the pass@k metric [17], defined as $\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$, where n denotes the total number of generated solutions per problem, c denotes the number of correct generations (i.e., those passing all test cases), and k denotes the number of generations considered (top-k). We measure the pass@k of the generated code before and after test suite augmentation and quantify the corresponding decrease.

We include GPT-4o, GPT-5, DeepSeek-R1, Gemini 2.5 Pro, and Claude Sonnet 4 in our evaluation, as they represent state-of-the-art and widely used LLMs [1]. All models are tested with default temperature and other parameters. Consistent with a prior study [84], we set n to 10 and refrain from using larger values such as 100 for pass@k, as this configuration provides a reasonable trade-off between computational cost and response time in practical settings. We measure pass@1, pass@5, and pass@10 both before and after test suite augmentation. For focal methods from MBJP, we use the dataset-provided prompts. For those from LeetCode, the problem descriptions serve as prompts. For non-standalone focal methods, we follow ComplexCodeEval by generating docstrings of focal

methods using LLMs and incorporating the generated docstrings into the prompt template for code generation. Specifically, we use Qwen3-235B-A22B for docstring generation, which differs from the evaluated models to avoid data leakage. For each focal method, code generation is performed using five LLMs, with ten solutions generated per LLM. Conducting experiments on all 1,035 focal methods would therefore require substantial time and computational resources. Consequently, we randomly selected 50 standalone and 50 non-standalone focal methods with generated killing test cases for evaluation. Using this balanced set of 100 focal methods, we assess the code generation performance of different LLMs.

Table 3. Pass@k Results Before and After Test Suite Augmentation

LLM	pass@1	pass@5	pass@10	pass@1*	pass@5*	pass@10*	pass@1↓	pass@5↓	pass@10↓	
GPT-4o	0.7820	0.9244	0.9400	0.4160	0.6165	0.6600	0.3660	0.3079	0.2800	
GPT-5	0.7680	0.8838	0.9000	0.4820	0.5993	0.6200	0.2860	0.2845	0.2800	
DeepSeek-R1	**0.8220**	**0.9473**	**0.9600**	0.4800	0.6540	**0.7200**	0.3420	0.2933	0.2400	S
Gemini 2.5 Pro	0.7980	0.8983	0.9000	**0.5320**	**0.6656**	0.6800	0.2660	0.2327	0.2200	
Claude Sonnet 4	0.7900	0.9161	0.9200	0.4740	0.6637	0.7000	0.3160	0.2524	0.2200	
GPT-4o	0.4220	0.5190	0.5400	0.2760	0.3913	0.4000	0.1460	0.1277	0.1400	
GPT-5	**0.5220**	**0.5921**	**0.6000**	0.2660	0.3429	0.3600	0.2560	0.2492	0.2400	
DeepSeek-R1	0.4220	0.5300	0.5600	**0.3020**	**0.4098**	**0.4200**	0.1200	0.1202	0.1400	NS
Gemini 2.5 Pro	0.4860	0.5694	0.5800	0.2760	0.3761	0.4000	0.2100	0.1933	0.1800	
Claude Sonnet 4	0.4240	0.5413	0.5600	0.2700	0.3912	**0.4200**	0.1540	0.1501	0.1400	

The * symbol indicates the pass@k values calculated after test suite augmentation.
S and NS represent standalone and non-standalone methods, respectively.

Results. Table 3 reports the pass@k (k = 1, 5, 10) results before and after augmentation, along with the corresponding decreases. Specifically, false positives here refer to LLM-generated code solutions that pass the initial test suite but are actually incorrect, as revealed by the augmented test suite. The drop in pass@k quantifies how many such false positives were mitigated. Clearly, for both standalone and non-standalone focal methods, pass@k (k = 1, 5, 10) decreases after augmentation. For standalone methods, the average drops across five LLMs in pass@1, pass@5, and pass@10 are 0.3152, 0.2742, and 0.2480, respectively. For non-standalone methods, the corresponding drops are 0.1772, 0.1681, and 0.1680. For standalone methods, the pass@k scores of all five LLMs are consistently higher than those for non-standalone methods, largely because the contextual information in non-standalone methods is more complex and may be insufficient to support effective code generation. Across the five LLMs, the pass@k (k = 1, 5, 10) values vary, and the bold entries in the table indicate the highest scores. For standalone methods, DeepSeek-R1 outperforms the other LLMs before test suite augmentation, whereas Gemini 2.5 Pro achieves the best results after augmentation. For non-standalone methods, GPT-5 obtains higher pass@k scores than the other LLMs prior to augmentation, while DeepSeek-R1 delivers the highest code generation performance among the evaluated models after augmentation.

The correctness of newly generated test cases is largely determined automatically based on whether they pass on the focal methods. These test cases pass on the focal methods but fail when executed on LLM-generated implementations, enabling differential testing to identify errors in LLM-generated code. The observed drop in pass@k is thus not attributable to incorrectly generated test cases. Additionally, during manual inspection of the execution results of augmented test suites, we observed overfitted generated test cases in isolated cases, which invoke local variables of the focal method. The pass@k values reported in Table 3 are calculated after excluding these overfitted test cases.

Answer for RQ3: After test suite augmentation, the pass@k (k = 1, 5, 10) values decrease across all evaluated LLMs, with Gemini 2.5 Pro and DeepSeek-R1 achieving the highest code generation performance for standalone and non-standalone methods, respectively.

5 Discussion

5.1 Ablation Study

Approach. ChatUniTest [18] proposes a framework of generation, validation, and repair for LLM-based test case generation, and many previous studies [7, 18, 21, 33, 47, 61, 68, 69, 78, 85, 90] have demonstrated the importance of test case repair. Consequently, we do not conduct an ablation study on test case repair and instead focus our analysis on the three augmentation steps implemented in RAGTIME. We use the 100 randomly selected focal methods from RQ3 to conduct an ablation study, which is sufficient to demonstrate the importance of each step. We remove one step at a time to create variants of our augmentation process.

Table 4. Average Augmented Mutation Score and Test Suite Size

Approach	Standalone		Non-Standalone	
	Augmented MS	Augmented TSS	Augmented MS	Augmented TSS
Full (Step1 + Step2 + Step3)	**88.16**	29.89	**96.95**	**17.82**
Full - Step1	85.80	**32.39**	94.79	16.55
Full - Step2	82.86	16.82	93.79	14.00
Full - Step3	87.03	24.11	94.91	15.57

MS stands for Mutation Score. TSS denotes Test Suite Size.

Results. Table 4 presents the average augmented mutation scores across the selected focal methods, with the highest values highlighted in bold. The row "*Full (Step1 + Step2 + Step3)*" denotes the use of all three steps for test suite augmentation. The row "*Full - Step1*" represents removing Step 1 while keeping both Step 2 and Step 3, whereas the rows "*Full - Step2*" and "*Full - Step3*" represent removing Step 2 and Step 3, respectively. As shown in Table 4, the augmented mutation score drops after removing each of the three steps. However, the decrease is more pronounced when Step 2 is removed compared to Step 1 or Step 3. This highlights the importance of Step 2, which performs test suite augmentation sequentially for each survived mutant. Moreover, omitting Step 3 yields the smallest reduction in mutation score; however, the augmented mutation score remains lower. This underscores the importance of RAG in leveraging generated mutant-killing test cases to eliminate non-killed mutants.

Table 4 also presents the average sizes of augmented test suites. For standalone methods, the removal of Step 1 produces an augmented test suite size that is marginally larger than that obtained when all three steps are applied. This demonstrates the role of Step 1 in reducing the size of augmented test suites by generating test cases targeting code lines where mutation is applied, which can kill multiple related mutants that originate from these lines. Furthermore, RAGTIME's optimization of test suite size can help reduce test execution cost, particularly in comparison with approaches such as AlphaCode [50], which typically generate around 200 test cases for CodeContests problems.

5.2 Evaluation of Sub-Approaches in SUNG

Approach. To understand the strengths of the six sub-approaches in SUNG for generating survived mutants, we calculate the proportion of survived mutants produced by each, using the initial test

suite. We analyze 100 randomly selected focal methods from RQ3, as evaluating all 1,035 would incur substantial cost.

Table 5. Proportion of Survived Mutants Generated by Each Sub-Approach

Focal Methods	Sub-Approach 1	Sub-Approach 2	Sub-Approach 3	Sub-Approach 4	Sub-Approach 5	Sub-Approach 6	
Standalone	12.16%	9.61%	9.88%	25.73%	14.63%	**27.98%**	Mean
	13.42%	6.83%	9.09%	**25.66%**	11.74%	25.00%	Median
Non-Standalone	15.61%	11.41%	10.16%	20.56%	12.06%	**30.20%**	Mean
	15.32%	11.27%	10.00%	20.38%	10.00%	**26.42%**	Median

Results. Table 5 presents the analysis results, with the highest values highlighted in bold. For each focal method, we calculate the proportion of survived mutants from each sub-approach. We then compute the average and median proportion for each sub-approach across all selected focal methods. Thus, the proportions in the '*Mean*' rows of Table 5 do not sum to 100%. Clearly, the boundary condition sub-approach (6th) generates the highest number of survived mutants. The fault localization sub-approach (4th) also contributes significantly to the generation of survived mutants. The results from applying the initial test suite suggest that boundary conditions within focal methods are less thoroughly tested. This highlights the need for testers to place greater emphasis on corner cases within source methods.

```
// Inferred program state: "Join Arguments: It joins all processed arguments into a single string"
.collect(Collectors.joining(" "));
==>
.collect(Collectors.joining(",")); // Mutant: Use ',' as delimiter
```

Fig. 11. Example of an inferred program state and a mutant generated based on that state. The example is taken from spring-cloud-deployer-2.9.2.

Note that the behavior model sub-approach contributes, on average, 12.16% of survived mutants for standalone methods and 15.61% for non-standalone methods. This highlights the non-negligible role of this sub-approach in generating mutants that expose inadequacies in test suites for assessing program behavior. Although this sub-approach may introduce minor syntactic or control-flow variations, this is not necessarily observed in all cases. Fig. 11 presents an example of a mutant generated by the behavior model sub-approach that does more than introduce minor syntactic or control-flow variations.

5.3 Quality of SUNG-Generated Mutants

Approach. We manually analyze the proportion of equivalent and unrealistic mutants produced by SUNG. Detecting equivalent mutants is fundamentally undecidable [15], and sound and complete identification of unrealistic mutants, which are unlikely to occur in real-world code, remains an open problem requiring semantic or historical analysis. Manually analyzing all mutants generated from 1,035 focal methods is labor-intensive. Consistent with prior studies [48, 76], which assess mutant equivalence using sampled subsets, we manually identify unrealistic and equivalent mutants on the randomly selected 100 focal methods from RQ3. Two authors independently analyze mutants from the selected focal methods. Inter-rater agreement is assessed using the AC1 coefficient [35, 79]. For standalone methods, the AC1 coefficients are 0.9289 for unrealistic mutants and 0.9922 for equivalent mutants. For non-standalone methods, the corresponding AC1 values are 0.9481 and

0.9854, respectively. All AC1 coefficients indicate almost perfect agreement. Disagreements are resolved through discussion to reach consensus.

```
for (EnumDescriptorProto ed : file.getEnumTypeList()) {
    EnumElement enumer = toEnum(ed);
    types.add(enumer); ==> types.add(null); // Mutant: Adding null instead of enumer
}
```

Fig. 12. Example of a classified unrealistic mutant generated from aws-glue-schema-registry-1.1.19.

Results. For standalone methods, the average proportions of unrealistic and equivalent mutants are 3.25% and 4.71%, respectively, while the corresponding proportions for non-standalone methods are 4.02% and 4.26%. Moreover, identifying unrealistic mutants perfectly remains challenging and relies on extensive programming experience. Fig. 12 presents an example of a mutant deemed unrealistic because developers typically do not insert null values into lists of schema elements.

5.4 Cost of Running SUNG and RAGTIME

High computational cost is a well-known challenge in mutation testing. For example, the baseline PIT can require more than a day to complete mutation testing for a single focal method, largely due to timeouts caused by infinite loops during mutant execution. Similarly, the augmentation pipeline may incur high end-to-end *latency*, reaching two hours or more for focal methods with more lines of code. Such methods typically involve a higher number of survived mutants and require the generation and execution of more test cases. In general, the primary sources of *latency* arise from two factors: the unstable response time of the DeepSeek API and the execution of newly generated test cases on focal methods and the corresponding survived mutants. Nevertheless, several practical optimizations are feasible, including asynchronous LLM prompting and test case execution, as well as parallel mutant execution, which could substantially reduce overall latency.

Although the augmentation pipeline may incur high latency, a high degree of parallel execution across focal methods can substantially reduce the total *time* required to perform augmentation for 1,035 focal methods. In this study, the average per-focal-method token usage is approximately 320k tokens, including both input and output tokens. Owing to the relatively low cost of the DeepSeek API, the average per-focal-method cost is approximately USD 0.059. Furthermore, the augmented test suites significantly strengthen source code testing and support a range of software engineering tasks. For example, they help reduce false positives in code generation evaluation and can be used for regression testing to guard against unintended changes. Their reusability across multiple scenarios therefore provides a meaningful return on investment (ROI).

5.5 Sensitivity to Initial Test Suites

The augmentation pipeline is not sensitive to the quality of the initial test suite. Weak initial test suites leave many survived mutants and yield low initial mutation scores, thereby providing substantial room for augmentation. For example, for standalone methods with an initial mutation score below 30%, the average mutation score improvement is 39.24%. In contrast, strong initial test suites leave fewer survived mutants. Although our study focuses on test suite augmentation and assumes a non-empty initial test suite, the pipeline can be readily extended to scenarios with empty test suites by treating all generated mutants as survived.

5.6 Threats to Validity

Construct validity. The main threat to construct validity is the presence of equivalent mutants, which ideally should not be included in the calculation of mutation score. However, detecting

equivalent mutants is undecidable [15]. We follow previous studies [45, 88] that do not specifically handle equivalent mutants. In the test suite augmentation process, both test case generation and test case repair have a limited number of iterations, preventing endless loops in cases where a survived mutant is equivalent. Additionally, we removed all duplicate mutants and any mutants that are identical to the original focal method from the mutants generated by SUNG when calculating the mutation score.

Internal validity. The main threat to internal validity lies in data leakage. Although the dataset we adopted may have been used during the pre-training stage of LLMs, our study generates new mutants and new test cases, neither of which exist in existing projects, thereby mitigating this threat. Another threat arises in code generation evaluation, where we hypothesize the adopted focal methods as ground-truth implementations. The test cases pass on these focal methods but fail when executed on LLM-generated code, forming differential testing to identify errors in the generated code. While the ground-truth implementations may be incorrect, the focal methods selected from widely used datasets (i.e., MBJP and LeetCode problems) and GitHub repositories help mitigate this threat.

External validity. This threat primarily stems from the dataset used in our study. Our analysis and conclusions may not generalize to all projects or application scenarios. However, our evaluation, based on 1,035 focal methods sourced from 526 competition problems and 27 open-source GitHub projects, helps mitigate this threat.

6 Conclusions and Future Work

In this paper, we propose an approach that integrates mutation testing with test suite augmentation to enhance the effectiveness of test suites using LLMs. Our semantic-level mutation testing enables the generation of survived mutants, which in turn guide targeted test case generation. Evaluation on both standalone and non-standalone methods demonstrates that the proposed approach effectively improves test suite mutation scores. Moreover, the observed decrease in pass@k across all evaluated LLMs underscores the approach's effectiveness in reducing false positives arising from insufficient test cases in code generation evaluation.

In future work, we plan to explore sharing additional information between SUNG and RAGTIME beyond survived-mutant-related information to further improve performance. We also intend to investigate the joint optimization of mutant generation and test case generation, allowing each component to iteratively improve the other through co-training. In this study, we primarily aim to demonstrate effectiveness, and we plan to investigate optimizations to reduce the latency of the augmentation pipeline.

Data Availability

Our code and data are available at [3].

Acknowledgments

We are grateful to the anonymous reviewers for their constructive suggestions that helped improve the quality of the paper. This work was supported in part by the National Natural Science Foundation of China (Grants 62572226, 624B2067, and 62172205), the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), and the Jiangsu Natural Science Foundation (Grant BK20231402).

References

[1] [n. d.]. Design Arena. https://www.designarena.ai/
[2] [n. d.]. JaCoCo. https://www.jacoco.org/jacoco/

[3] [n. d.]. Supplementary Material. https://doi.org/10.5281/zenodo.17279914.

[4] [n. d.]. Understand. http://www.scitools.com

[5] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[6] Iftekhar Ahmed, Rahul Gopinath, Caius Brindescu, Alex Groce, and Carlos Jensen. 2016. Can testedness be effectively measured?. In *Proceedings of the 2016 24th ACM sigsoft international symposium on foundations of software engineering*. 547–558. https://doi.org/10.1145/2950290.2950324

[7] Juan Altmayer Pizzorno and Emery D Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2897–2919. https://doi.org/10.1145/3729398

[8] James H Andrews, Lionel C Briand, Yvan Labiche, and Akbar Siami Namin. 2006. Using mutation analysis for assessing and comparing testing coverage criteria. *IEEE Transactions on Software Engineering* 32, 8 (2006), 608–624. https://doi.org/10.1109/TSE.2006.83

[9] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2022. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868* (2022).

[10] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[11] Yushi Bai, Jiajie Zhang, Xin Lv, Linzhi Zheng, Siqi Zhu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. Longwriter: Unleashing 10,000+ word generation from long context llms. *arXiv preprint arXiv:2408.07055* (2024).

[12] Richard Baker and Ibrahim Habli. 2012. An empirical evaluation of mutation testing for improving the test quality of safety-critical software. *IEEE Transactions on Software Engineering* 39, 6 (2012), 787–805. https://doi.org/10.1109/TSE.2012.56

[13] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. 2018. A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)* 51, 3 (2018), 1–39. https://doi.org/10.1145/3182657

[14] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).

[15] Timothy A Budd and Dana Angluin. 1982. Two notions of correctness and their relation to testing. *Acta informatica* 18, 1 (1982), 31–45. https://doi.org/10.1007/BF00625279

[16] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. 2008. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, Vol. 8. 209–224.

[17] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[18] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576.

[19] Yiqun T Chen, Rahul Gopinath, Anita Tadakamalla, Michael D Ernst, Reid Holmes, Gordon Fraser, Paul Ammann, and René Just. 2020. Revisiting the relationship between fault detection, test adequacy criteria, and test set size. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 237–249. https://doi.org/10.1145/3324884.3416667

[20] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. Pit: a practical mutation testing tool for java. In *Proceedings of the 25th international symposium on software testing and analysis*. 449–452. https://doi.org/10.1145/2931037.2948707

[21] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C Desmarais. 2024. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* 171 (2024), 107468. https://doi.org/10.1016/j.infsof.2024.107468

[22] Renzo Degiovanni and Mike Papadakis. 2022. μbert: Mutation testing using pre-trained language models. In *2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 160–169.

[23] Richard A DeMillo, A Jefferson Offutt, et al. 1991. Constraint-based automatic test data generation. *IEEE Transactions on Software Engineering* 17, 9 (1991), 900–910. https://doi.org/10.1109/32.92910

[24] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-collaboration code generation via chatgpt. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–38. https://doi.org/10.1145/3672459

[25] Jia Feng, Jiachen Liu, Cuiyun Gao, Chun Yong Chong, Chaozheng Wang, Shan Gao, and Xin Xia. 2024. ComplexCodeEval: A Benchmark for Evaluating Large Code Models on More Complex Code. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1895–1906. https://doi.org/10.1145/3691620.3695552

[26] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).

[27] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419. https://doi.org/10.1145/2025113.2025179

[28] Gordon Fraser and Andrea Arcuri. 2012. Whole test suite generation. *IEEE Transactions on Software Engineering* 39, 2 (2012), 276–291. https://doi.org/10.1109/TSE.2012.14

[29] Milos Gligoric, Alex Groce, Chaoqiang Zhang, Rohan Sharma, Mohammad Amin Alipour, and Darko Marinov. 2013. Comparing non-adequate test suites using coverage criteria. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis*. 302–313. https://doi.org/10.1145/2483760.2483769

[30] Milos Gligoric, Alex Groce, Chaoqiang Zhang, Rohan Sharma, Mohammad Amin Alipour, and Darko Marinov. 2015. Guidelines for coverage-based comparisons of non-adequate test suites. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24, 4 (2015), 1–33. https://doi.org/10.1145/2660767

[31] Rahul Gopinath, Carlos Jensen, and Alex Groce. 2014. Code coverage for suite evaluation by developers. In *Proceedings of the 36th International Conference on Software Engineering*. 72–82. https://doi.org/10.1145/2568225.2568278

[32] Giovanni Grano, Fabio Palomba, and Harald C Gall. 2019. Lightweight assessment of test-case effectiveness using source-code-quality indicators. *IEEE Transactions on Software Engineering* 47, 4 (2019), 758–774. https://doi.org/10.1109/TSE.2019.2903057

[33] Siqi Gu, Chunrong Fang, Quanjun Zhang, Fangyuan Tian, and Zhenyu Chen. 2024. TestART: Improving LLM-based Unit Test via Co-evolution of Automated Generation and Repair Iteration. *arXiv e-prints* (2024), arXiv–2408.

[34] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13. https://doi.org/10.1145/3597503.3623306

[35] Kilem Li Gwet. 2008. Computing inter-rater reliability and its variance in the presence of high agreement. *Brit. J. Math. Statist. Psych.* 61, 1 (2008), 29–48. https://doi.org/10.1348/000711006X126600

[36] Hojae Han, Jaejin Kim, Jaeseok Yoo, Youngwon Lee, and Seung-won Hwang. 2024. ArchCode: Incorporating Software Requirements in Code Generation with Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13520–13552.

[37] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938* (2021).

[38] Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2019. Re-factoring based program repair applied to programming assignments. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 388–398. https://doi.org/10.1109/ASE.2019.00044

[39] Ali Reza Ibrahimzada, Yang Chen, Ryan Rong, and Reyhaneh Jabbarvand. 2023. Automated bug generation in the era of large language models. *arXiv preprint arXiv:2310.02407* (2023).

[40] Laura Inozemtseva and Reid Holmes. 2014. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th international conference on software engineering*. 435–445.

[41] Konrad Jamrozik, Gordon Fraser, Nikolai Tillmann, and Jonathan De Halleux. 2012. Augmented dynamic symbolic execution. In *Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering*. 254–257. https://doi.org/10.1145/2351676.2351716

[42] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).

[43] René Just, Darioush Jalali, Laura Inozemtseva, Michael D Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing?. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 654–665. https://doi.org/10.1145/2635868.2635929

[44] Mohammad Abdullah Matin Khan, M Saiful Bari, Do Long, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. 2024. Xcodeeval: An execution-based large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6766–6805.

[45] Jinhan Kim, Juyoung Jeon, Shin Hong, and Shin Yoo. 2022. Predictive mutation analysis via the natural language channel in source code. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 4 (2022), 1–27. https://doi.org/10.1145/3510417

[46] Marinos Kintis, Mike Papadakis, Andreas Papadopoulos, Evangelos Valvis, Nicos Malevris, and Yves Le Traon. 2018. How effective are mutation testing tools? An empirical analysis of Java mutation testing tools with manual analysis and real faults. *Empirical Software Engineering* 23, 4 (2018), 2426–2463. https://doi.org/10.1007/s10664-017-9582-5

[47] Michael Konstantinou, Renzo Degiovanni, Jie M Zhang, Mark Harman, and Mike Papadakis. 2025. YATE: The Role of Test Repair in LLM-Based Unit Test Generation. *arXiv preprint arXiv:2507.18316* (2025).
[48] Benjamin Kushigian, Samuel J Kaufman, Ryan Featherman, Hannah Potter, Ardi Madadi, and René Just. 2024. Equivalent mutants in the wild: Identifying and efficiently suppressing equivalent mutants for Java programs. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 654–665.
[49] Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. 2023. Taco: Topics in algorithmic code generation dataset. *arXiv preprint arXiv:2312.14852* (2023).
[50] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097. https://doi.org/10.1126/science.abq1158
[51] Jinwei Liu, Chao Li, Rui Chen, Shaofeng Li, Bin Gu, and Mengfei Yang. 2025. STRUT: Structured Seed Case Guided Unit Test Generation for C Programs using LLMs. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 2113–2135. https://doi.org/10.1145/3728970
[52] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2024).
[53] Larry J. Morell. 1990. A theory of fault-based testing. *IEEE Transactions on Software Engineering* 16, 8 (1990), 844–857. https://doi.org/10.1109/32.57623
[54] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
[55] AJV Offutt. 1989. Automatic test data generation. (1989).
[56] Haidar Osman, Mircea Lungu, and Oscar Nierstrasz. 2014. Mining frequent bug-fix code changes. In *2014 Software Evolution Week-IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering (CSMR-WCRE)*. IEEE, 343–347. https://doi.org/10.1109/CSMR-WCRE.2014.6747191
[57] Xianfei Ou, Cong Li, Yanyan Jiang, and Chang Xu. 2024. The Mutators Reloaded: Fuzzing Compilers with Large Language Model Generated Mutation Operators. ASPLOS. https://doi.org/10.1145/3622781.3674171
[58] Shuyin Ouyang, Jie M Zhang, Mark Harman, and Meng Wang. 2024. An Empirical Study of the Non-determinism of ChatGPT in Code Generation. *ACM Transactions on Software Engineering and Methodology* (2024).
[59] Carlos Pacheco and Michael D Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*. 815–816. https://doi.org/10.1145/1297846.1297902
[60] Rongqi Pan, Taher A Ghaleb, and Lionel C Briand. 2024. LTM: Scalable and Black-box Similarity-based Test Suite Minimization based on Language Models. *IEEE Transactions on Software Engineering* (2024).
[61] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. 2024. Multi-language unit test generation using llms. *arXiv preprint arXiv:2409.03093* (2024).
[62] Terence Parr. 2013. The definitive ANTLR 4 reference. *The Definitive ANTLR 4 Reference* (2013), 1–326.
[63] Goran Petrović, Marko Ivanković, Gordon Fraser, and René Just. 2023. Please fix this mutant: How do developers resolve mutants surfaced during code review?. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 150–161. https://doi.org/10.1109/ICSE-SEIP58684.2023.00019
[64] Ajitha Rajan. 2006. Automated requirements-based test case generation. *ACM SIGSOFT Software Engineering Notes* 31, 6 (2006), 1–2. https://doi.org/10.1145/1218776.1218799
[65] José Miguel Rojas, Thomas D White, Benjamin S Clegg, and Gordon Fraser. 2017. Code defenders: crowdsourcing effective tests and subtle mutants with a mutation testing game. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 677–688. https://doi.org/10.1109/ICSE.2017.68
[66] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 951–971.
[67] Ana B Sánchez, José A Parejo, Sergio Segura, Amador Durán, and Mike Papadakis. 2024. Mutation Testing in Practice: Insights from Open-Source Software Developers. *IEEE Transactions on Software Engineering* (2024).
[68] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* (2023).
[69] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinícius Carvalho Lopes. 2024. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 313–322.
[70] Zhao Tian, Junjie Chen, Qihao Zhu, Junjie Yang, and Lingming Zhang. 2022. Learning to construct better mutation faults. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[71] Frank Tip, Jonathan Bell, and Max Schäfer. 2024. LLMorpheus: Mutation Testing using Large Language Models. *arXiv preprint arXiv:2404.09952* (2024).
[72] Eric K Tokuda, Cesar H Comin, and Luciano da F Costa. 2022. Revisiting agglomerative clustering. *Physica A: Statistical mechanics and its applications* 585 (2022), 126433. https://doi.org/10.1016/j.physa.2021.126433
[73] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).
[74] Michele Tufano, Jason Kimko, Shiya Wang, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, and Denys Poshyvanyk. 2020. Deepmutation: A neural mutation tool. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*. 29–32.
[75] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. Learning how to mutate source code from bug-fixes. In *2019 IEEE International conference on software maintenance and evolution (ICSME)*. IEEE, 301–312.
[76] Bo Wang, Mingda Chen, Ming Deng, Youfang Lin, Mark Harman, Mike Papadakis, and Jie M Zhang. 2024. A Comprehensive Study on Large Language Models for Mutation Testing. *arXiv preprint arXiv:2406.09843* (2024).
[77] Guancheng Wang, Qinghua Xu, Lionel C Briand, and Kui Liu. 2025. Towards More Effective Fault Detection in LLM-Based Unit Test Generation. *arXiv preprint arXiv:2506.02954* (2025).
[78] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1258–1268. https://doi.org/10.1145/3691620.3695501
[79] Nahathai Wongpakaran, Tinakon Wongpakaran, Danny Wedding, and Kilem L Gwet. 2013. A comparison of Cohen's Kappa and Gwet's AC1 when calculating inter-rater reliability coefficients: a study conducted with personality disorder samples. *BMC medical research methodology* 13, 1 (2013), 61.
[80] Weixiang Yan, Haitian Liu, Yunkun Wang, Yunzhe Li, Qian Chen, Wen Wang, Tingyu Lin, Weishan Zhao, Li Zhu, Hari Sundaram, et al. 2023. Codescope: An execution-based multilingual multitask multidimensional benchmark for evaluating llms on code understanding and generation. *arXiv preprint arXiv:2311.08588* (2023).
[81] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *arXiv preprint arXiv:2404.04966* (2024).
[82] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, et al. 2024. On the evaluation of large language models in unit test generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1607–1619.
[83] Xin Yin, Chao Ni, Xinrui Li, Liushan Chen, Guojun Ma, and Xiaohu Yang. 2025. Enhancing LLM's Ability to Generate More Repository-Aware Unit Tests Through Precise Contextual Information Injection. *arXiv preprint arXiv:2501.07425* (2025).
[84] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12. https://doi.org/10.1145/3597503.3623316
[85] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726. https://doi.org/10.1145/3660783
[86] Junwei Zhang, Xing Hu, Xin Xia, Shing-Chi Cheung, and Shanping Li. 2025. Automated Unit Test Generation via Chain of Thought Prompt and Reinforcement Learning from Coverage Feedback. *ACM Transactions on Software Engineering and Methodology* (2025). https://doi.org/10.1145/3745765
[87] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. *arXiv preprint arXiv:2401.07339* (2024).
[88] Peng Zhang, Yang Wang, Xutong Liu, Zeyu Lu, Yibiao Yang, Yanhui Li, Lin Chen, Ziyuan Wang, Chang-ai Sun, Xiao Yu, et al. 2023. Assessing effectiveness of test suites: what do we know and what should we do? *ACM Transactions on Software Engineering and Methodology* (2023). https://doi.org/10.1145/3635713
[89] Yucheng Zhang and Ali Mesbah. 2015. Assertions are strongly correlated with test suite effectiveness. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 214–224. https://doi.org/10.1145/2786805.2786858
[90] Zhe Zhang, Xingyu Liu, Yuanzhang Lin, Xiang Gao, Hailong Sun, and Yuan Yuan. 2024. LLM-based Unit Test Generation via Property Retrieval. *arXiv preprint arXiv:2410.13542* (2024).
[91] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. 2023. A survey of large language models for code: Evolution, benchmarking, and future trends. *arXiv preprint arXiv:2311.10372* (2023).
[92] Hong Zhu, Patrick AV Hall, and John HR May. 1997. Software unit test coverage and adequacy. *Acm computing surveys (csur)* 29, 4 (1997), 366–427. https://doi.org/10.1145/267580.267590

Received 2025-10-10; accepted 2026-02-17